Aligning DNA reads to detect variants

Minimum-cost alignment and the actual edit operations for 4×10^8 reads of 150 bases

Roll no.	Name
24I315	Eswari S
25I462	Chamilayah R
25I464	Kaniga V
25I465	Purnimaa V S
25I466	Vijith S

1 · Problem modelling

Eswari S

- **Input:** reference region R (length m) and read Q (length n) over {A,C,G,T}; m ≈ n ≈ 150.
- **Costs:** match 0, substitution 1, insertion 1, deletion 1.
- **Output:** minimum cost and one alignment as edit operations (M, S, I, D). Each S/I/D is a candidate variant.
- **Objective:** minimise total cost over all alignments.
- **Constraints:** every base used once, in order; no reordering.
- **Class:** optimisation (cost + witness). Classical problem: **edit distance / global sequence alignment** (Wagner–Fischer, Needleman–Wunsch).

Scale
One read: 151 × 151 = 22,801 cells.
One run: 4×10^8 reads ≈ **9.1×10^{12}** cell updates.
Two 10^6-base genomes: 10^{12} cells (≈ 4 TB).

2 · Data structure analysis

Structure	Access / update	Fill time	Space (150 × 150)	Space ($10^6 \times 10^6$)	Edit ops?
2-D array (chosen)	O(1)	Θ(mn)	22,801 cells ≈ 22 KB (fits cache)	≈ 4 TB	Yes, traceback O(m+n)
Hash-map memo	O(1) expected, bigger constant	Θ(mn)	same entries, tens of bytes each	far above 4 TB	Yes
Two rolling rows	O(1)	Θ(mn)	≈ 300 bytes	≈ 8 MB	**No** (path lost)
Recursion stack, no table	–	$\Theta(5.83^n)$	O(m+n)	O(m+n)	Only by redoing work
Linear-memory version (Q4)	O(1)	≤ 2mn	O(n)	≈ 8 MB	Yes

Choice. The 2-D array: O(1) access, small enough for cache at 150 bases, and it keeps the path the lab needs. Rolling rows lose the path; at 10^6 bases we switch to the linear-memory version.

3 · Technique: Dynamic Programming

Purnimaa V S

```
ed(i,j) = min( ed(i-1,j-1) + s(ri,qj),
               ed(i-1,j)   + 1,          // deletion
               ed(i,j-1)   + 1 )         // insertion
ed(i,0) = i,   ed(0,j) = j,   s = 0 if ri = qj else 1
```

Length 4 × 4
25 distinct subproblems
481 recursive calls
subproblem (2,2) solved **13 times**

- **Optimal substructure.** The last column of an optimal alignment is a match/substitution, a deletion or an insertion. Remove it: what is left must be optimal for the shorter prefixes, otherwise a cheaper prefix alignment plus the removed column would beat the original (cut-and-paste).
- **Overlapping subproblems.** Only (m+1)(n+1) distinct subproblems, but the recursion visits them over and over.

4a · Rejected: Greedy and Divide and Conquer

Greedy — counterexample

Reference	Read	Greedy	True min	Greedy ops
GATTACA	GCATGCA	3	3	S S S (coincidence)
ACGT	**CGT**	**4**	**1**	S S S D

Optimal: delete A, then 3 matches. Greedy commits to A/C at the first base and can never undo it: 4 false variants instead of 1. Greedy-choice property fails.

Divide and Conquer — subproblems overlap

i \ j	0	1	2	3	4
0	63	88	32	8	1
1	88	63	25	7	1
2	32	25	**13**	5	1
3	8	7	5	3	1
4	1	1	1	1	1

Times each subproblem is solved on two strings of length 4: 481 calls for 25 subproblems. Last-column split: $T(n) = 3T(n-1) + O(1) = \Theta(3^n)$, not the $a \cdot T(n/b)$ form. Middle split needs the unknown crossing column in the read, so all n+1 candidates overlap.

4b · Rejected: Backtracking and Branch and Bound

Backtracking (plain recursion)

n	Table cells	Recursion calls
4	25	481
7 (the instance)	64	72,958
10	121	12,146,179
11	144	67,570,078
150	22,801	$\approx 10^{113.7}$

$T(i,j) = 1 + T(i-1,j-1) + T(i-1,j) + T(i,j-1)$. Growth rate $3 + 2\sqrt{2} \approx 5.83$ per extra base: unrunnable at 150 bases.

Branch and Bound (bound: cost so far + length gap)

n	B&B nodes	Table cells
6	243	49
8	1,050	81
10	3,416	121

Equal-length strings make the length-gap term 0, so the bound is just the cost so far. Seeded with the greedy cost it prunes a lot (55 nodes on the instance) but on random reads is 28× the table at n = 10 and keeps growing about 1.8–2× per base. A tight bound would need the table itself.

> Both fail the same test: the table method does (m+1)(n+1) units of work, they do exponentially many.

5 · Algorithm

Kaniga V

```
D[i][0] ← i·DEL ;  D[0][j] ← j·INS
for i ← 1..m:  for j ← 1..n:
    diag ← D[i-1][j-1] + (0 if R[i]=Q[j] else SUB)
    up   ← D[i-1][j]   + DEL
    left ← D[i][j-1]   + INS
    D[i][j] ← min(diag, up, left)
// traceback from (m,n); order: diagonal, deletion, insertion
while i>0 or j>0:
    if i,j>0 and D[i][j] = D[i-1][j-1] + s : emit M/S ; i--, j--
    elif i>0 and D[i][j] = D[i-1][j] + DEL : emit D ; i--
    else                                   : emit I ; j--
```

State: D[i][j] = minimum cost to align R[1..i] with Q[1..j].
Recurrence: slide 4.
Answer: D[m][n]; path from the traceback.

6 · Trace: GATTACA vs GCATGCA

	-	G	C	A	T	G	C	A
-	**0**	1	2	3	4	5	6	7
G	1	**0**	1	2	3	4	5	6
A	2	1	**1**	1	2	3	4	5
T	3	2	2	**2**	1	2	3	4
T	4	3	3	3	**2**	2	3	4
A	5	4	4	3	3	**3**	3	3
C	6	5	4	4	4	4	**3**	4
A	7	6	5	4	5	5	4	**3**

Rows: reference prefixes; columns: read prefixes. Shaded = the traceback path.

- **D[2][2]** (A vs C): diag 0+1 = 1; up 2; left 2 → 1
- **D[3][4]** (T vs T): diag 1+0 = 1; up 3; left 3 → 1
- **D[7][7]** (A vs A): diag 3+0 = 3; up 5; left 5 → 3

```
Ref : G A T T A C A
      |     |   | |
Read: G C A T G C A
Ops : M S S M S M M   cost 3
```

Variants: pos 2 A>C, pos 3 T>A, pos 5 A>G. The table has **4** co-optimal paths (MSSMSMM, MIMDMSMM, MIMMDSMM, MIMMSDMM).

Minimum cost = 3

7 · Proof of correctness

Theorem. After the fill, D[i][j] = ed(i,j) for all i, j.

- **Base (i+j smallest):** the only alignment of i bases with nothing deletes all i, so ed(i,0) = i; likewise ed(0,j) = j. The initialisation stores exactly this.
- **Step (induction on i+j):** an optimal alignment of R[1..i], Q[1..j] ends in one of three column types. Removing it leaves an optimal alignment of the smaller prefixes (cut-and-paste), worth D[i−1][j−1], D[i−1][j] or D[i][j−1] by hypothesis.
- **Loop invariant:** those three cells have smaller index sum and are computed earlier in row-major order, so they are final when D[i][j] is computed. The min over the three options covers every alignment, so D[i][j] = ed(i,j). ∎
- **Traceback invariant:** the operations emitted so far cost exactly D[m][n] − D[i][j]. A test always holds (D[i][j] was a minimum of the three), each step lowers i+j, and at (0,0) the emitted cost is D[m][n]. ∎

8 · Complexity and worst case

Chamilayah R

	Time	Space
Table method (best = worst)	$\Theta(mn)$ always: 3 candidates × (m+1)(n+1) states	$\Theta(mn)$; $\Theta(n)$ cost only
Practical (banded, cost ≤ k)	$O(kn)$: k = 10 → ≈ 3,150 cells vs 22,801	$O(kn)$
Plain recursion	$\Theta((3+2\sqrt{2})^n/\sqrt{n})$	$O(m+n)$
Linear-memory version	≤ 2mn	$O(n)$

Fleet scale: $4 \times 10^8 \times 22{,}801 \approx 9.1 \times 10^{12}$ cell updates ≈ 2.5 core-hours at an assumed 10^9 updates/s/core.

Worst-case input: input-independent; the stress case is A^n vs C^n (no zero-cost cell). Measured at n = 1600 (same 2.57 M cells each):

Input	Min cost	Fill time (ms)
identical	0	426
mutated (~3%)	47	389
random pair	846	799
A^n vs C^n	1600	441

Same operation count; time differences are Python integer allocation, not algorithmic. The recursion makes the same 481 calls for any 4 × 4 input.

Q4 · Linear memory for $10^6 \times 10^6$

- Full table: 10^{12} cells ≈ **4 TB**. Keeping two rows is O(n) but forgets the path.
- **Recover the path by divide and conquer (Hirschberg):**
- 1. Split the reference at the middle row.
- 2. Forward last-row scores for the top half; backward last-row scores for the bottom half (reversed strings).
- 3. The optimal path crosses the middle row at the column k minimising forward[k] + backward[n−k].
- 4. Recurse on the two halves; stop at one reference base.

Space: O(n) → a few MB.
Time: level ℓ costs $mn/2^{\ell}$, total ≤ 2mn (≈ 2×10^{12} updates).
Measured (n = 1000): full table 25,513 KB, linear version 87 KB.

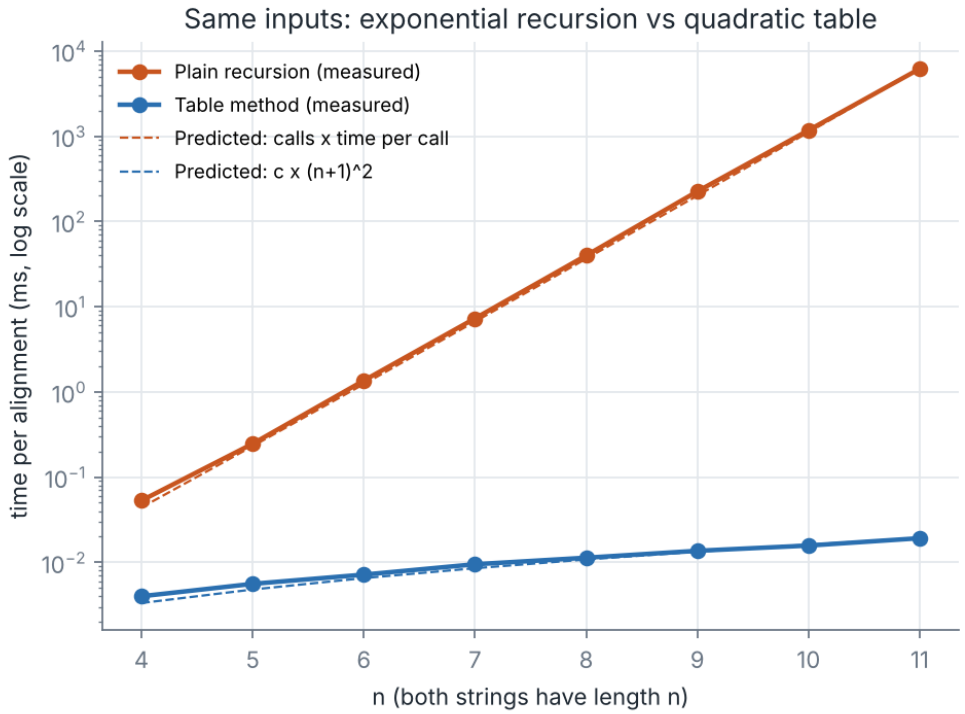
Q5 · Affine gaps: open 3, extend 1

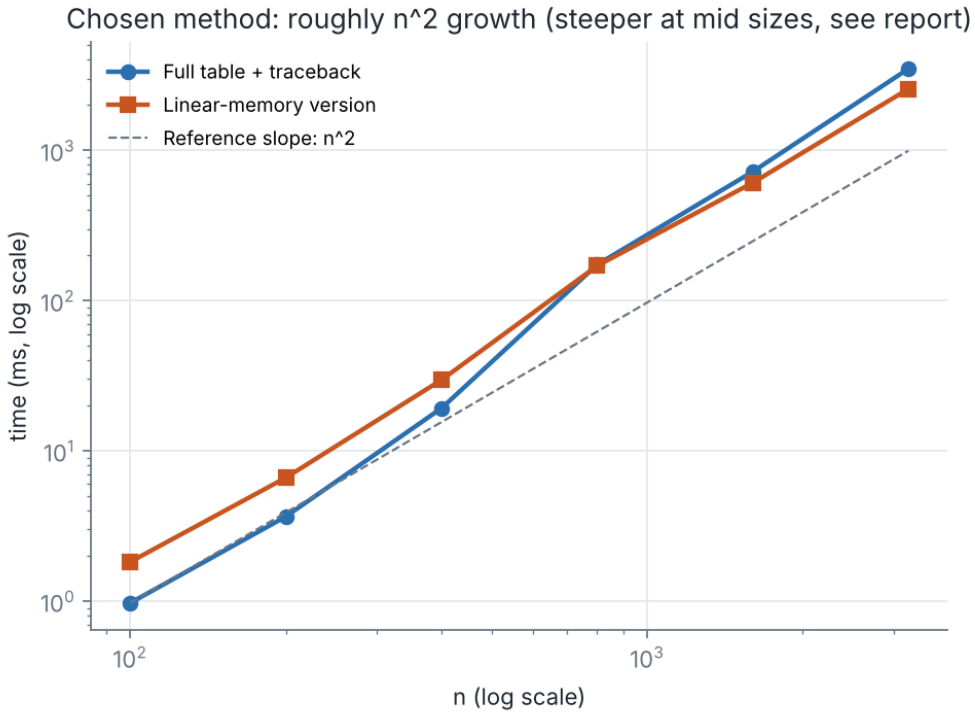
```
M[i][j] = s(ri,qj) + min(M, X, Y)[i-1][j-1]
X[i][j] = min( M[i-1][j]+3,  X[i-1][j]+1,  Y[i-1][j]+3 )   // deletion
Y[i][j] = min( M[i][j-1]+3,  Y[i][j-1]+1,  X[i][j-1]+3 )   // insertion
answer  = min(M, X, Y)[m][n]      gap of length k costs 3 + (k−1)
```

- Why three tables: the cost of one more gap base depends on whether the previous column was a gap, so the state must include the last column type.
- **Extra storage:** 3 × 22,801 = 68,403 cells per read (two extra tables, 3×); time per cell also ≈ 3×. Linear memory: three rows, still O(n).
- **Instance:** cost 3 with MSSMSMM; the other three co-optimal alignments now cost 7, so substitutions win uniquely.
- **Technique still holds:** still DP, states (i, j, last type); the other four stay rejected. General gap cost γ(k) would be Θ(mn(m+n)).

9 · Implementation and empirical analysis

Vijith S





- Python, no compilation; code for the table method and the plain recursion.
- Same inputs, n = 4..11: recursion 0.053 → 6,175 ms (log-linear, matches calls × time per call); table 0.004 → 0.019 ms.
- At n = 11 the recursion is ≈ 320,000× slower.
- Scaling n = 100..3200: ≈ n^2 overall, but µs per cell rises 0.095 → 0.34 (cache and allocation in Python); the linear-memory version even ends up faster there (ratio 0.73).
- 500 random pairs cross-checked: all methods agree on cost; alignments reproduce both strings.

10 · Real-world reference

- **BWA-MEM** (Li, 2013) and **minimap2** (Li, 2018): production read aligners that end in this same dynamic program.
- **Their version:** find seeds first (FM-index / minimizers) to decide where a read belongs, then run a **banded, SIMD** dynamic program with **affine gaps** (local alignment in minimap2, via ksw2) only near the seeds.
- **Our version:** global, unbanded, unit-cost alignment of a read against a given region.
- Related building blocks: Hirschberg 1975 (linear memory), Gotoh 1982 (affine gaps), Myers–Miller 1988 (both), Myers 1999 (bit-parallel, used in Edlib).

Summary

Question	Answer
Q1 Minimum cost / ops	3; M S S M S M M (4 co-optimal alignments)
Q2 Fast aligner	GATTACA: 3 (coincidence). ACGT vs CGT: 4, true 1
Q3 Recursion	481 calls for 4 × 4 vs 25 table cells
Q4 Linear memory	Hirschberg: O(n) space, ≤ 2mn time
Q5 Affine gaps	3 tables (M, X, Y): 3× storage, same technique